SAT Optimization: A Graph Neural Network with Attention Approach

Speaker: Chuang-Chieh Lin (Joseph)
NTOU, TW

a joint work with

Kuang-Neng Chen
NTOU, TW

Chen-Yo Chung
NTOU, TW

Chih-Chieh Hung*
NCHU, TW

@CMCT 2026

Outline

Problem & Motivation

Methodology

Experiments

Conclusion

Why MaxSAT matters

Maximum Satisfiability (MaxSAT)

Given a CNF formula, find a Boolean assignment that **maximizes the number of satisfied clauses**.

Why MaxSAT matters

Maximum Satisfiability (MaxSAT)

Given a CNF formula, find a Boolean assignment that **maximizes the number of satisfied clauses**.

$$\text{E.g.,} \qquad \varphi_4 = (x_1) \ \wedge \ (x_2) \ \wedge \ (\neg x_1 \vee \neg x_2).$$

Why MaxSAT matters

Maximum Satisfiability (MaxSAT)

Given a CNF formula, find a Boolean assignment that **maximizes the number of satisfied clauses**.

$$\text{E.g.,} \qquad \varphi_4 = (x_1) \ \wedge \ (x_2) \ \wedge \ (\neg x_1 \vee \neg x_2).$$

- Fundamental NP-hard opt. problem.
- A lot of practical applications.
- Classical solvers are powerful, but rely heavily on hand-crafted heuristics.

Why MaxSAT matters

Maximum Satisfiability (MaxSAT)

Given a CNF formula, find a Boolean assignment that **maximizes the number of satisfied clauses**.

$$\text{E.g.,} \qquad \varphi_4 = (x_1) \ \wedge \ (x_2) \ \wedge \ (\neg x_1 \vee \neg x_2).$$

- Fundamental NP-hard opt. problem.
- A lot of practical applications.
- Classical solvers are powerful, but rely heavily on hand-crafted heuristics.

Optimization target

For clauses $c_1, \ldots, c_m$,

$$\arg\max_{x \in \{\pm 1\}^n} \#\{j : c_j \text{ is satisfied by } x\}.$$

Equivalently, minimize #unsatisfied clauses.

Why look at neural approaches?

Traditional MaxSAT solvers

- Complete solvers: optimal but may be expensive.
- Incomplete solvers: faster, but may get stuck in local optima.
- Performance depends strongly on problem structure.

Existing neural solvers (torchSAT)

- Differentiable optimization enables GPU acceleration.
- But simple aggregation treats all neighboring clauses uniformly.
- This may ignore **which constraints are most critical**.

Why look at neural approaches?

Traditional MaxSAT solvers

- Complete solvers: optimal but may be expensive.
- Incomplete solvers: faster, but may get stuck in local optima.
- Performance depends strongly on problem structure.

Existing neural solvers (torchSAT)

- Differentiable optimization enables GPU acceleration.
- But simple aggregation treats all neighboring clauses uniformly.
- This may ignore **which constraints are most critical**.

Main question

Can we explicitly exploit the **bipartite graph structure** of CNF formulas and let the model **adaptively focus** on *important* clauses?

Our main idea and contributions

1. **Graph modeling:** represent a CNF formula as a variable-clause bipartite graph.
2. **GraphSAT:** use learnable directional message passing instead of fixed summation.
3. **GraphAttentionSAT:** add cross-attention so variables can focus on critical clauses.
4. **Theory:** show per-iteration complexity

$$\mathcal{O}(|E|d + (n+m)d^2).$$

5. **Empirics:** on larger instances, significantly improve solution quality over torchmSAT.

Our main idea and contributions

1. **Graph modeling:** represent a CNF formula as a variable-clause bipartite graph.
2. **GraphSAT:** use learnable directional message passing instead of fixed summation.
3. **GraphAttentionSAT:** add cross-attention so variables can focus on critical clauses.
4. **Theory:** show per-iteration complexity

$$\mathcal{O}(|E|d + (n+m)d^2).$$

5. **Empirics:** on larger instances, significantly improve solution quality over torchmSAT.

The architecture is more expressive than uniform aggregation, yet remains scalable on sparse SAT instances.

Outline

Problem & Motivation

Methodology

Experiments

Conclusion

From CNF to a bipartite graph

Graph representation

For a CNF formula, define

$$G = (V, C, E),\ E = \{(v_i, c_j) : v_i \text{ appears in } c_j\}.$$

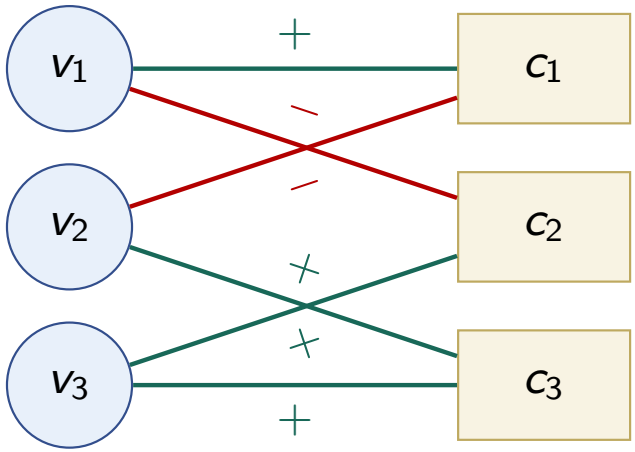
Here $V = \{v_1, \ldots, v_n\}$ are variables and $C = \{c_1, \ldots, c_m\}$ are clauses.

Signed incidence matrix $W \in \mathbb{R}^{n \times m}$

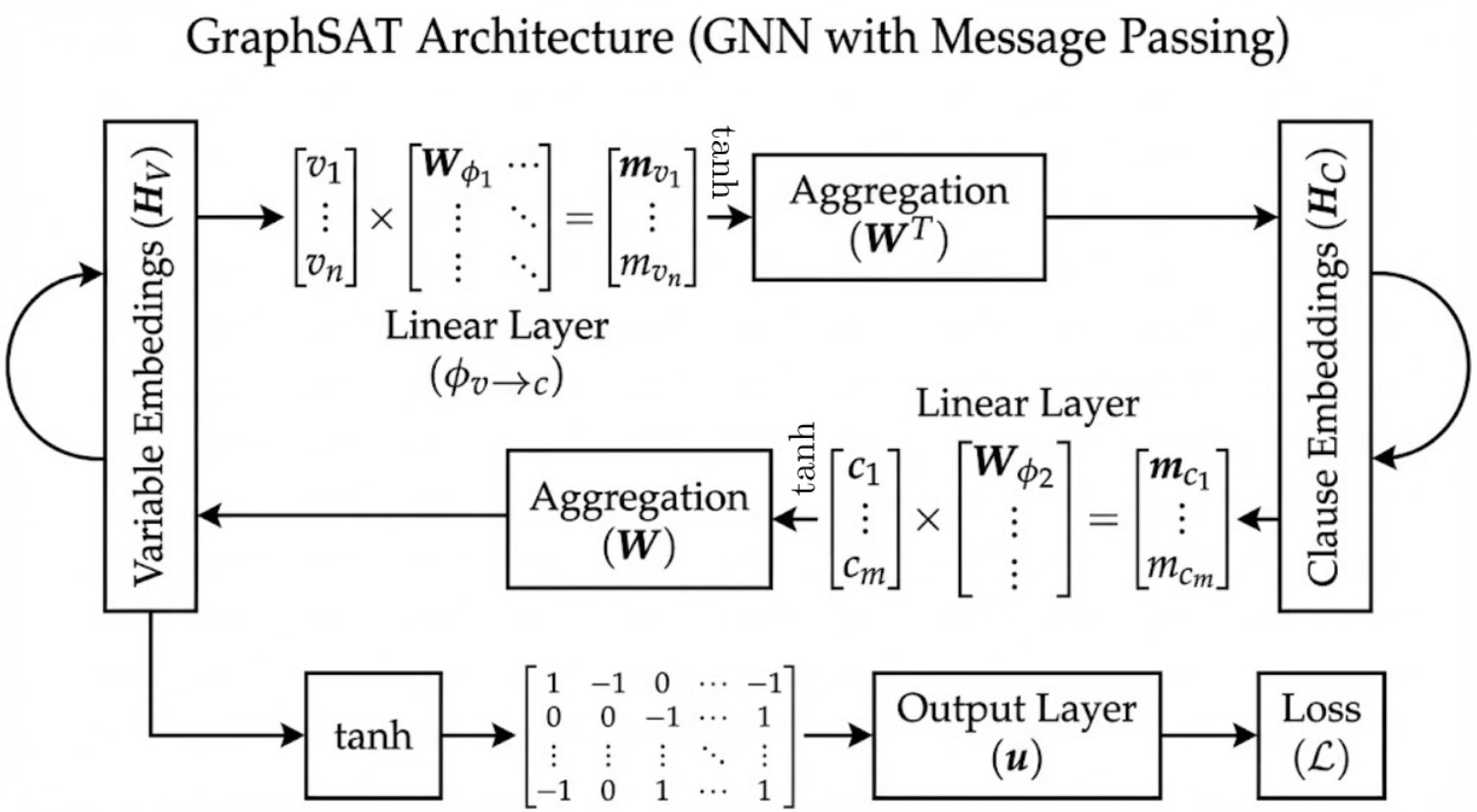
$$W_{ij} = \begin{cases} 1 & \text{positive occurrence,} \\ -1 & \text{negative occurrence,} \\ 0 & \text{not in } c_j. \end{cases}$$

So the graph records both connectivity and literal polarity.

Example: $c_1 = (v_1 \vee \neg v_2)$, $c_2 = (\neg v_1 \vee v_3)$, $c_3 = (v_2 \vee v_3)$



GraphSAT architecture overview



Variable and clause embeddings interact through directional message passing, then the output layer produces a continuous assignment optimized by the MaxSAT surrogate loss.

GraphSAT: learnable message passing

$W_{\phi_1} = \phi_{v\to c},\ W_{\phi_2} = \phi_{c\to v} \in \mathbb{R}^{d\times d}$: learnable matrices.

Variable → clause

$$M_C = W^\top \tanh\big(\phi_{v\to c}(H_V)\big)$$

Variables send assignment tendencies to adjacent clauses through a learned transformation.

Clause → variable

$$M_V = W \tanh\big(\phi_{c\to v}(H_C)\big)$$

Clauses return feedback about which variable changes may improve satisfaction.

State update

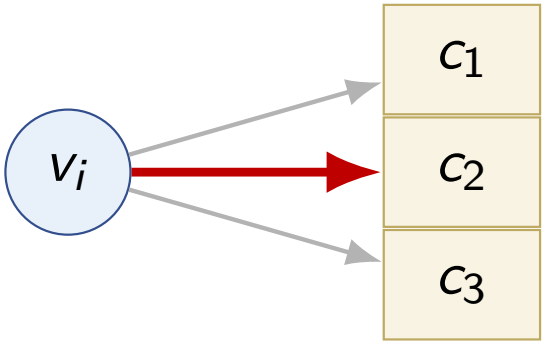
$$H_V^{(t+1)} = \tanh\left(H_V^{(t)} + M_V\right).$$

Why not uniform aggregation?

The model can learn **which interactions matter most**, rather than weighting every neighboring clause identically.

Why introduce attention?

- During optimization, not all clauses are equally important.
- Difficult or currently unsatisfied clauses should influence variable updates more strongly.
- This is analogous to the intuition behind classical solver heuristics: prioritize critical constraints.



attention weights adapt to clause importance

Goal of GraphAttentionSAT

Allow variables to **dynamically focus** on the most relevant clauses at the current iteration.

GraphAttentionSAT: masked cross-attention

$W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$: learnable projection matrices.

Attention construction

$$Q_V = H_V W_Q, \quad K_C = H_C W_K, \quad V_C = H_C W_V,$$

$$\text{Score} = \frac{Q_V K_C^\top}{\sqrt{d}} \in \mathbb{R}^{n \times m}.$$

Graph-aware masking

$$\text{Mask}(\text{Score})_{ij} = \begin{cases} \text{Score}_{ij}, & (v_i, c_j) \in E, \\ -\infty, & \text{otherwise.} \end{cases}$$

Only incident variable-clause pairs are allowed to interact.

Final aggregation

$$\text{Attn}(Q, K, V) = \text{softmax}(\text{Mask}(\text{Score}))V_C.$$

- connected clauses only;
- adaptive clause weighting;
- sparse-aware computation.

Differentiable loss for MaxSAT

Continuous assignment

The output layer produces $u \in [-1, 1]^n$, a continuous relaxation of Boolean assignments.

Clause unsatisfaction proxy

For clause c_j,

$$p_j = \prod_{v_i \in c_j} \frac{1 - W_{ij} u_i}{2}.$$

This serves as a differentiable proxy for clause unsatisfaction.

Optimization objective

$$\mathcal{L} = \sum_{j=1}^{m} p_j.$$

Minimizing $\mathcal{L}$ encourages the model to satisfy as many clauses as possible.

- end-to-end differentiable;
- GPU-friendly;
- instance-wise optimization.

Complexity analysis

GraphSAT per iteration

- Linear layers: $\mathcal{O}((n+m)d^2)$
- Sparse multiplications with $W, W^\top$: $\mathcal{O}(|E|d)$
- Nonlinearities and updates: $\mathcal{O}((n+m)d)$

$$\Rightarrow \mathcal{O}(|E|d + (n+m)d^2)$$

GraphAttentionSAT per iteration

- Q/K/V projections: $\mathcal{O}((n+m)d^2)$
- Sparse masked attention: $\mathcal{O}(|E|d)$
- Softmax and aggregation: $\mathcal{O}(|E|d)$

$$\Rightarrow \mathcal{O}(|E|d + (n+m)d^2)$$

Key point

Attention improves expressiveness without changing the **asymptotic per-iteration complexity**.

Outline

Problem & Motivation

Methodology

Experiments

Conclusion

Experimental setup

- Implemented in **PyTorch**; baseline comparison with **torchmSAT**.
- GPU-enabled environments, including NVIDIA T4 on Google Colab and local GPUs.
- Fixed setting for all models: **200 iterations**, hidden dimension $d = 64$.

Family	Data IDs	Variables/Clauses	Structural flavor
CB	0, 1	(176/290), (756/1,280)	small to moderate structured instances
GT	2, 3	(90/775), (380/7,050)	hierarchical constraints
PAR	4, 5	(210/4,011), (465/13,516)	intricate variable-clause dependencies
PHP	6, 7	(110/561), (420/4,221)	tight combinatorial constraints

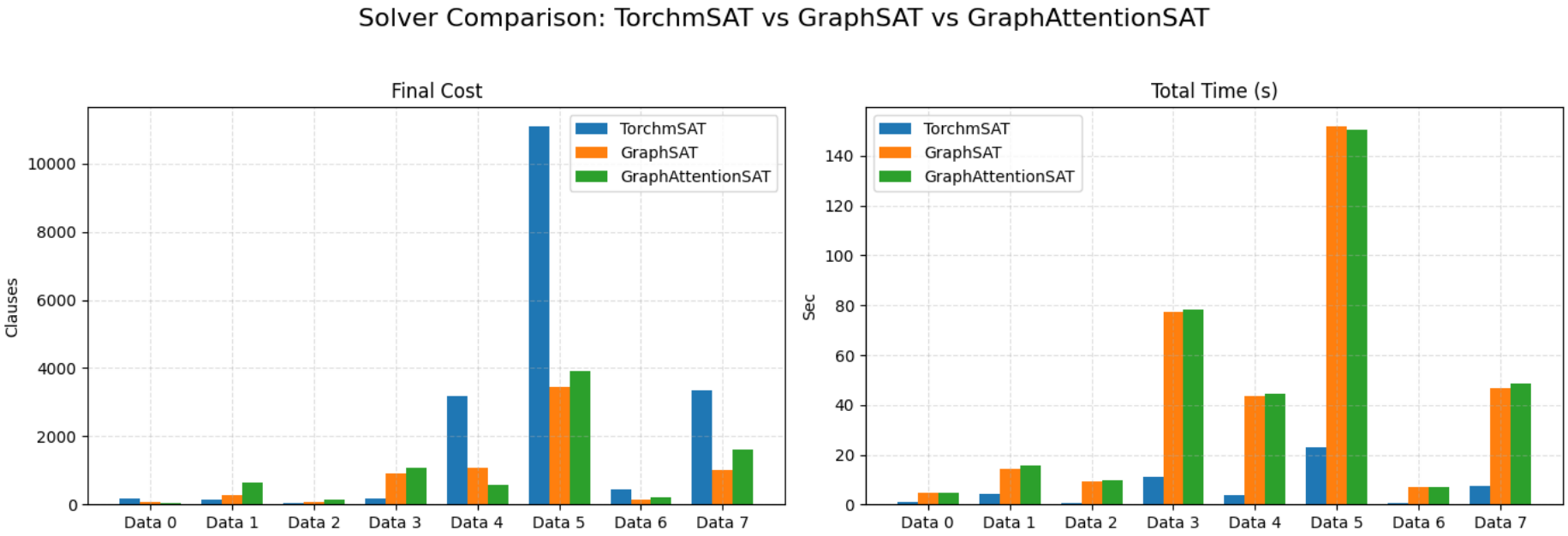
Experimental setup

Family	Data IDs	Variables/Clauses	Structural flavor
CB	0, 1	(176/290), (756/1,280)	small to moderate structured instances
GT	2, 3	(90/775), (380/7,050)	hierarchical constraints
PAR	4, 5	(210/4,011), (465/13,516)	intricate variable-clause dependencies
PHP	6, 7	(110/561), (420/4,221)	tight combinatorial constraints

Why these datasets?

They cover different graph structures, scales, and clause-variable dependency patterns.

Overall empirical comparison



Left: final cost (lower is better); Right: total running time (sec).

- GraphSAT and GraphAttentionSAT clearly improve solution quality on larger instances.

Solution quality: where do we gain the most?

Large-scale instances

Compared with torchmSAT, **GraphSAT** achieves substantial *cost* reduction:

- Data 3 (GT-20): $\approx$ **25%** lower;
- Data 4 (PAR-10): $\approx$ **72%** lower;
- Data 5 (PAR-15): $\approx$ **70%** lower;
- Data 7 (PHP-20): $\approx$ **56%** lower.

Interpretation

The gain is strongest when clause-variable dependencies are complex, especially in the **Parity** family.

Why?

Learnable message passing captures structure that uniform aggregation tends to miss.

Solution quality: where do we gain the most?

Large-scale instances

Compared with torchmSAT, **GraphSAT** achieves substantial *cost* reduction:

- Data 3 (GT-20): $\approx$ **25%** lower;
- Data 4 (PAR-10): $\approx$ **72%** lower;
- Data 5 (PAR-15): $\approx$ **70%** lower;
- Data 7 (PHP-20): $\approx$ **56%** lower.

Interpretation

The gain is strongest when clause-variable dependencies are complex, especially in the **Parity** family.

Why?

Learnable message passing captures structure that uniform aggregation tends to miss.

What happens on small instances?

- On Data 0–1, GraphSAT and GraphAttentionSAT can be worse than the baseline.
- A plausible explanation is a **capacity/generalization trade-off**.

Higher expressiveness helps most when the instance is structurally rich enough to need it.

Possible reasons

1. **Over-parameterization:** with $d = 64$, model capacity is large relative to graph size.
2. **Baseline simplicity:** torchmSAT may generalize better on trivial instances.
3. **Overhead not amortized:** richer updates cost more per iteration on small problems.

Runtime trade-off and the role of attention

Runtime trade-off

- GraphSAT and GraphAttentionSAT take roughly **7–8x** more time than the baseline.
- This is consistent with the richer computation implied by

$$\mathcal{O}(|E|d + (n+m)d^2).$$

Attention effect

- GraphAttentionSAT is usually close to GraphSAT.
- On these synthetic datasets, attention gives only marginal extra benefit.
- But it may help more on irregular industrial instances.

Quality-time trade-off.

Main empirical takeaways

- **Structured message passing works:** both proposed models outperform torchmSAT on larger instances.
- **PAR benchmark family instances are especially revealing:** the benefit is strongest when dependencies are intricate.
- **Attention is promising but not dominant here:** the main gain already comes from graph-aware message passing.
- **There is a clear quality-time trade-off:** better solutions come with higher computational overhead.

Broader implication

The learnable message-passing paradigm may be useful beyond MaxSAT, for other constraint satisfaction and combinatorial optimization problems.

Outline

Problem & Motivation

Methodology

Experiments

Conclusion

Conclusion

- We proposed **GraphSAT** and **GraphAttentionSAT** for MaxSAT optimization.
- The core ideas are:
  - bipartite graph modeling of CNF,
  - directional learnable message passing,
  - optional masked cross-attention.
- Theoretical analysis shows scalable per-iteration complexity:

$$\mathcal{O}(|E|d + (n+m)d^2).$$

- Experiments show strong gains on larger instances, while also revealing a meaningful runtime-quality trade-off.
- The learnable message-passing paradigm may be useful beyond MaxSAT,

Future directions

- Evaluate on standard MaxSAT benchmark suites.
- Study adaptive *hyperparameter tuning* across problem scales.
- Analyze optimization and *convergence* behavior more rigorously.
- Explore hybrid solvers combining neural guidance with classical MaxSAT techniques.
- Investigate whether attention becomes more valuable on *industrial instances*.

Thanks for your attention!

Q & A